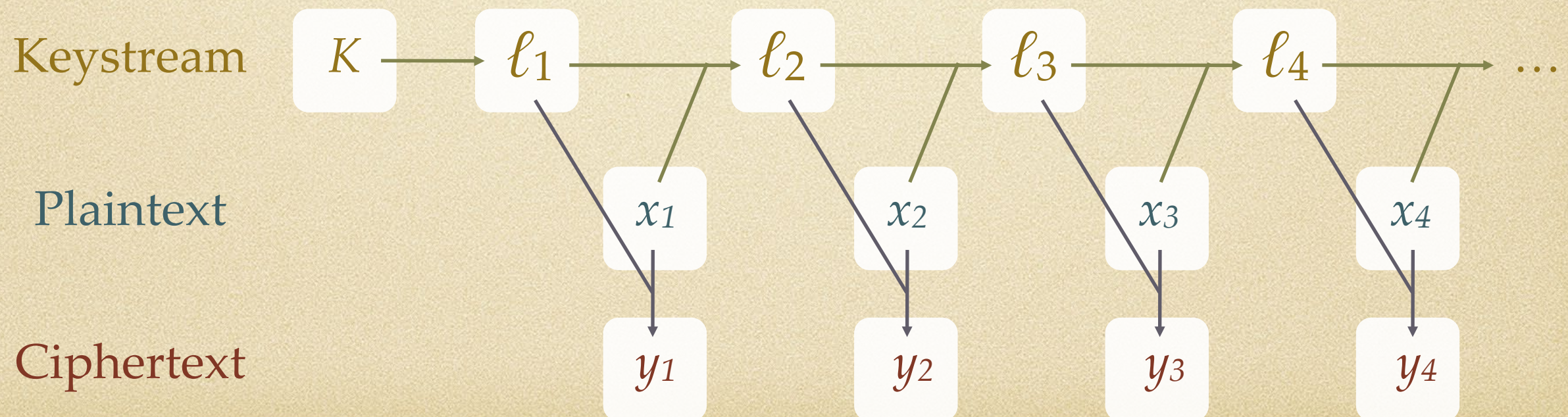


MATH3302/7332

Stream ciphers

Stream ciphers

- Encrypt one (letter / bit) at a time, using a **new key** at each step.
- The “stream” of keys is generated on the fly, and **may depend** on the preceding **plaintext**.
- The secret is the **initial key** K , often called the **seed**.



Autokey cipher

Using seed $K = 8$:

Plaintext (x_i)	r	e	n	d	e	z	v	o	u	s
	17	4	13	3	4	25	21	14	20	18
Keystream (ℓ_i)	8	17	4	13	3	4	25	21	14	20
	25	21	17	16	7	3	20	9	8	12
Ciphertext (y_i)	Z	V	R	Q	H	D	U	J	I	M

Stream ciphers

- Formally: $(\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathcal{L}, \mathcal{F}, \mathcal{E}, \mathcal{D})$
- $\mathcal{P}$ is the (finite) plaintext space
- $\mathcal{C}$ is the (finite) ciphertext space
- $\mathcal{K}$ is the (finite) keyspace, used for the seed
- $\mathcal{L}$ is the keystream alphabet, used for the encryption keys
- $\mathcal{E}, \mathcal{D}$ contain encryption and decryption rules:
For each $\ell \in \mathcal{L}$ we have $e_\ell \in \mathcal{E}$ and $d_\ell \in \mathcal{D}$ with
 $e_\ell : \mathcal{P} \rightarrow \mathcal{C}$, $d_\ell : \mathcal{C} \rightarrow \mathcal{P}$, and $d_\ell(e_\ell(x)) = x$ for every $x \in \mathcal{P}$

Stream ciphers

$\mathcal{F} = (f_1, f_2, \dots)$ is the **keystream generator**:

- For each $i \geq 1$, f_i is a function which generates the i th element of the keystream, $\ell_i \in \mathcal{L}$
- Each f_i is a function of the initial key K and the first $i-1$ plaintext characters

Types of stream ciphers

The keystream generator does **not need** to use the plaintext. This is just an option if you want it.

- **Synchronous** stream cipher:
Keystream is independent of the plaintext
(Depends only on the seed K)
- **Asynchronous** stream cipher:
Keystream is not independent of the plaintext

Periodic with period d : $\ell_{i+d} = \ell_i$ for all i
(Like the Vigenère cipher)

Autokey cipher

- $\mathcal{P} = \mathcal{C} = \mathcal{K} = \mathcal{L} = \mathbb{Z}_{26}$
- Keystream:
 $\ell_1 = K$ (the seed)
 $\ell_i = x_{i-1}$ (the previous plaintext letter), for $i \geq 2$
- Encryption and decryption:
 $e_\ell(x) = x + \ell, \quad d_\ell(y) = y - \ell$
- Non-periodic, asynchronous

Linear feedback shift register (LFSR)

- Works in **binary**: $\mathcal{P} = \mathcal{C} = \mathcal{L} = \mathbb{Z}_2 = \{ 0, 1 \}$
- Keystream generated by a **linear recurrence**
- Encryption and decryption:
 $e_\ell(x) = x + \ell, \quad d_\ell(y) = y + \ell$

Remember! In $\mathbb{Z}_2$ we always have $a + b = a - b$.

Linear feedback shift register (LFSR)

An m -stage LFSR uses a recurrence of degree m

Seed includes:

- First m keystream bits: $\ell_1, \ell_2, \dots, \ell_m \in \mathbb{Z}_2$
- Constants $c_0, c_1, \dots, c_{m-1} \in \mathbb{Z}_2$

Recurrence for subsequent keystream bits:

$$\ell_{i+m} = c_0\ell_i + c_1\ell_{i+1} + \dots + c_{m-1}\ell_{i+m-1}$$

Linear feedback shift register (LFSR)

Example: A 4-stage LFSR is given by:

$$\ell_1 = 1, \ell_2 = 0, \ell_3 = 1, \ell_4 = 1$$

$$\ell_{i+4} = \ell_i + \ell_{i+1} + \ell_{i+3} \text{ for } i = 1, 2, 3, \dots$$

Properties

Is a LFSR:

- (a) Synchronous or asynchronous?
- (b) Periodic or non-periodic?

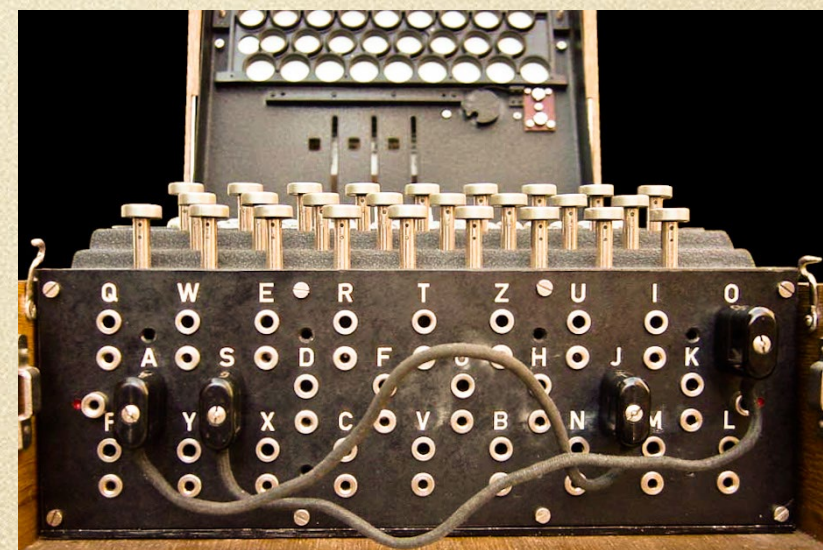
Cryptanalysis

Linear cipher $\Rightarrow$ **known plaintext** attack

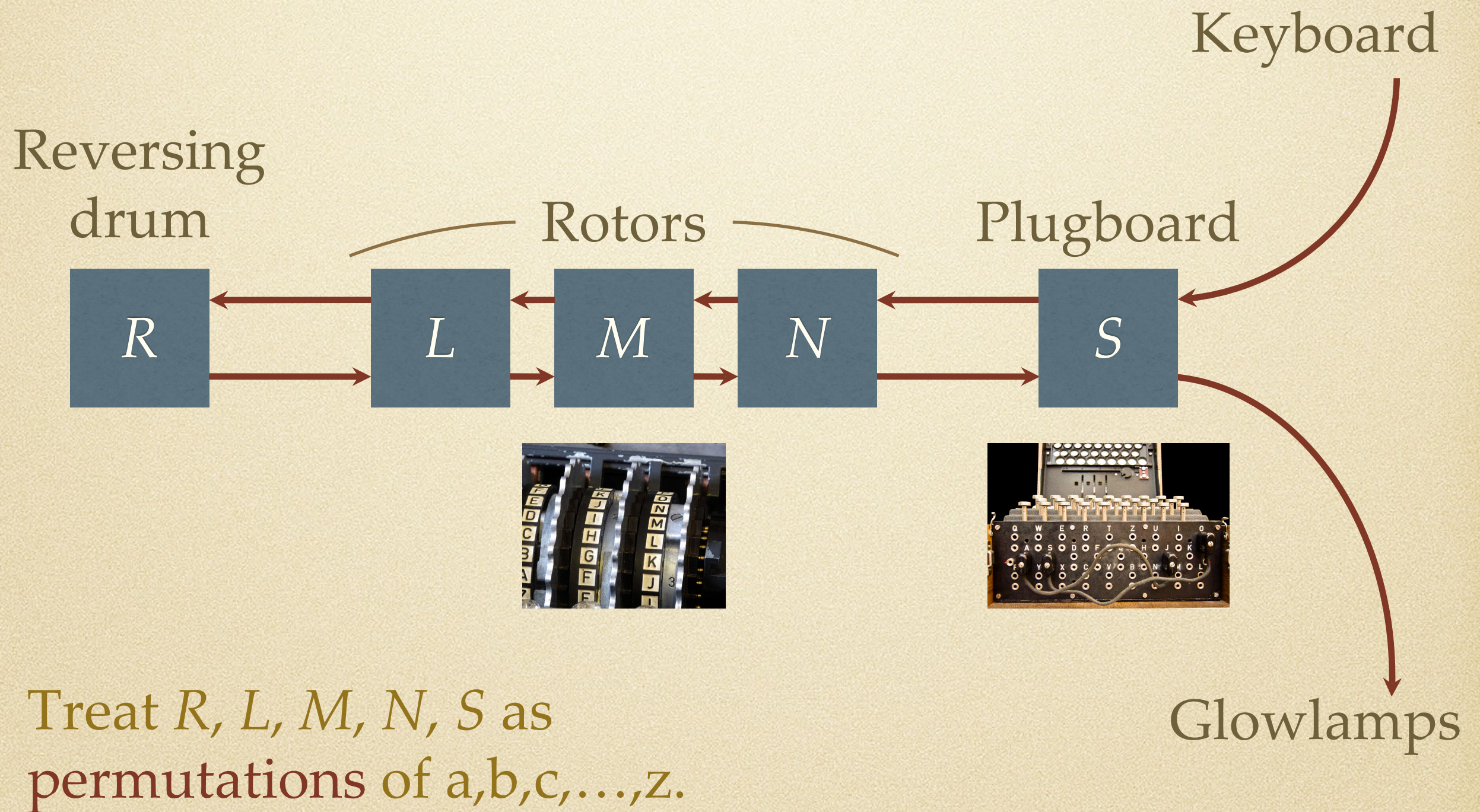
- Need $2m$ consecutive (plaintext, ciphertext) pairs
- Compute $2m$ consecutive keystream bits $\ell_1, \dots, \ell_{2m}$:
 $\ell_i = x_i + y_i$
- Obtain m linear equations for $\ell_{m+1}, \dots, \ell_{2m}$:
 $\ell_{i+m} = c_0\ell_i + c_1\ell_{i+1} + \dots + c_{m-1}\ell_{i+m-1}$
- Solve over $\mathbb{Z}_2$ for $c_0, \dots, c_{m-1}$
- Reconstruct the full keystream using the recurrence

Cryptanalysis

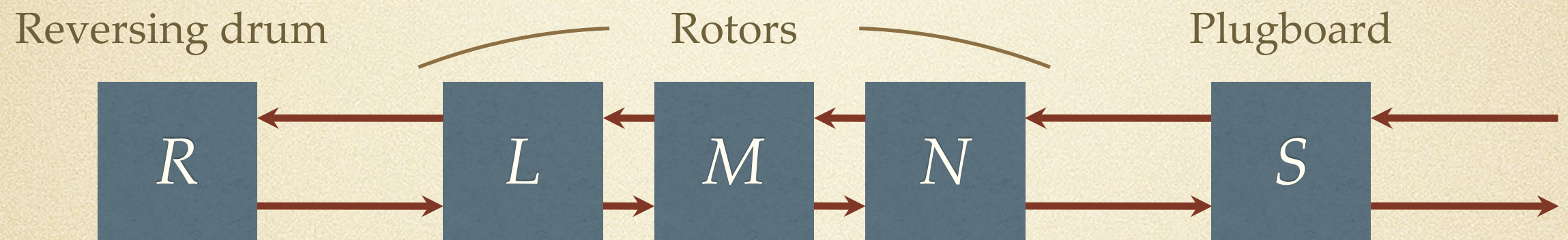
The Enigma



Enigma's components



Enigma's components



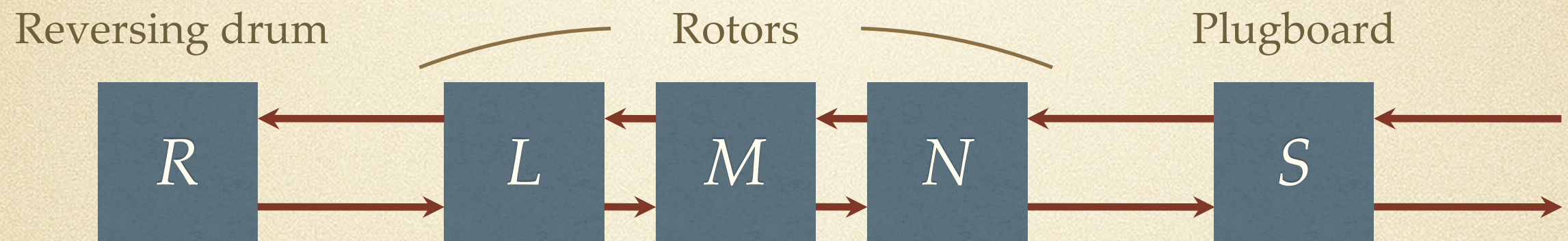
Reversing drum (*R*):

- Arranges all 26 letters into 13 pairs, and swaps each pair of letters
- $R^2 = 1$, and $R(x) \neq x$ for all x
- Hard-wired

Plugboard (*S*):

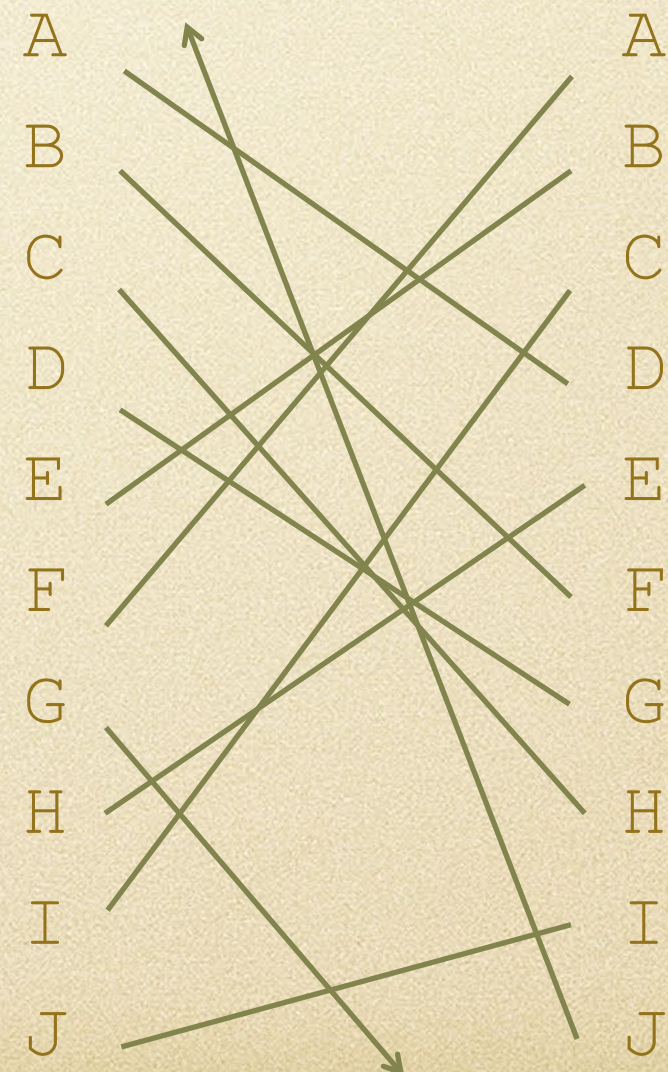
- Allows up to 6 pairs of letters to be swapped
- $S^2 = 1$
- Configured by the user

Enigma's components



Rotors (*L*, *M*, *N*):

- Arbitrary permutations of a,b,c,...,z
- Internal wiring fixed
- User sets initial rotations



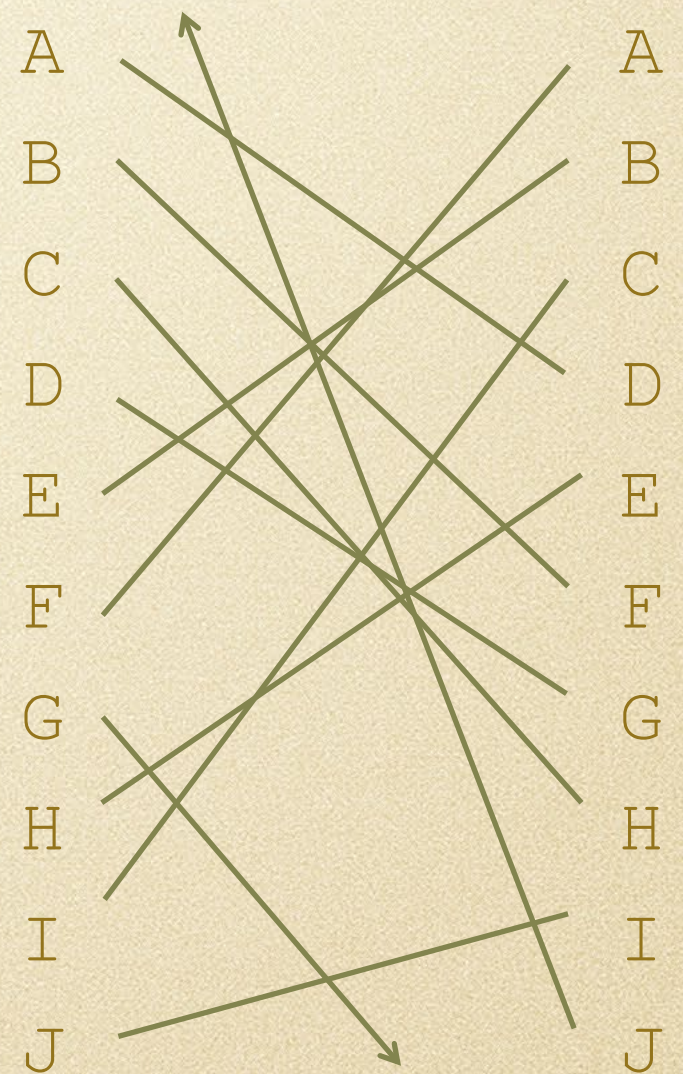
The keystream

Rotors turn on every keypress!

Like a clock:

- N rotates with every keypress
- M rotates each time N returns to position zero (every 26 keypresses)
- L rotates each time M returns to position zero (every 26×26 keypresses)

Periodic with period $26^3 = 17,576$



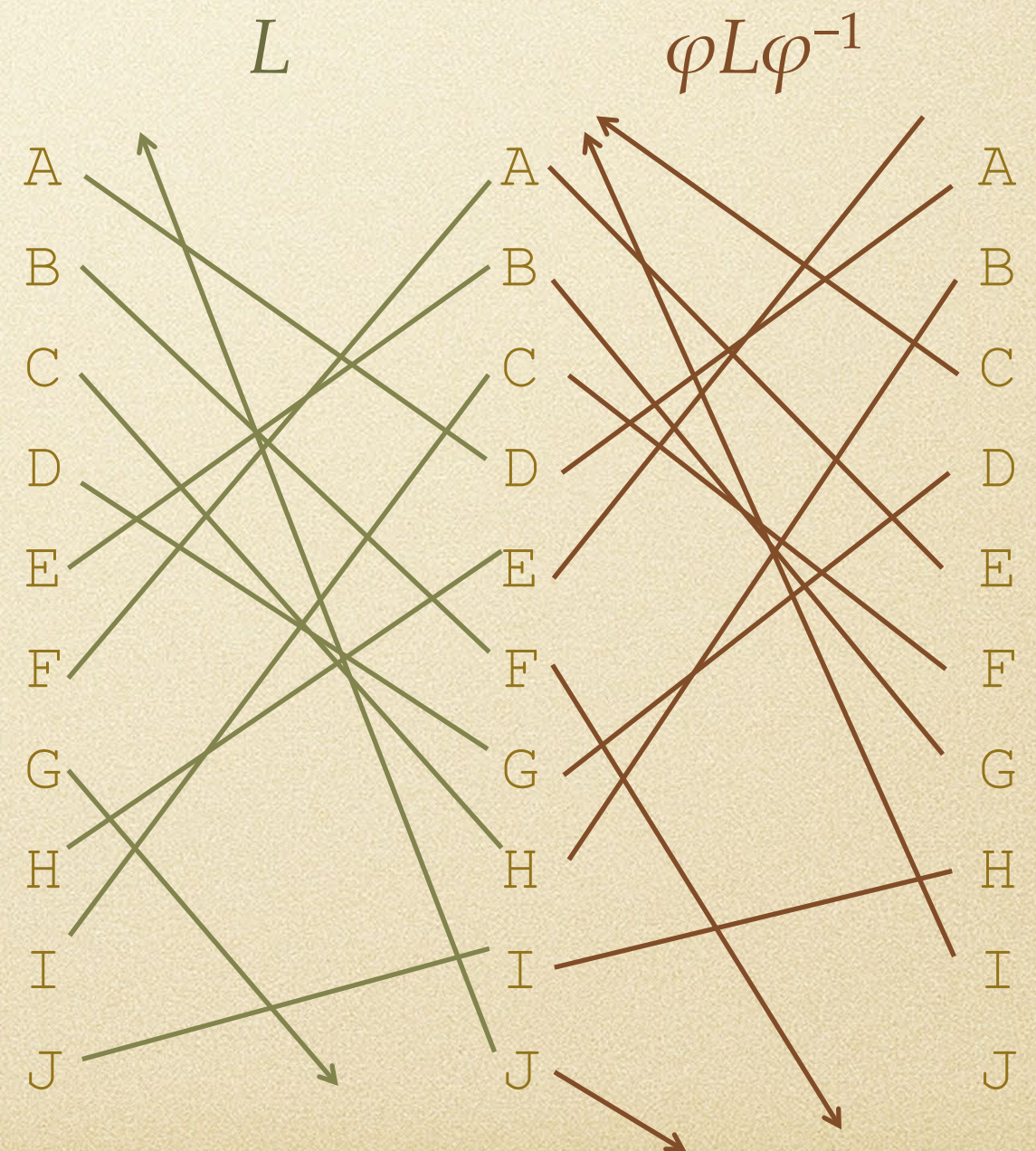
Rotor permutations

Define the shift $\varphi = (a\ b\ c\ d\ \dots\ y\ z)$

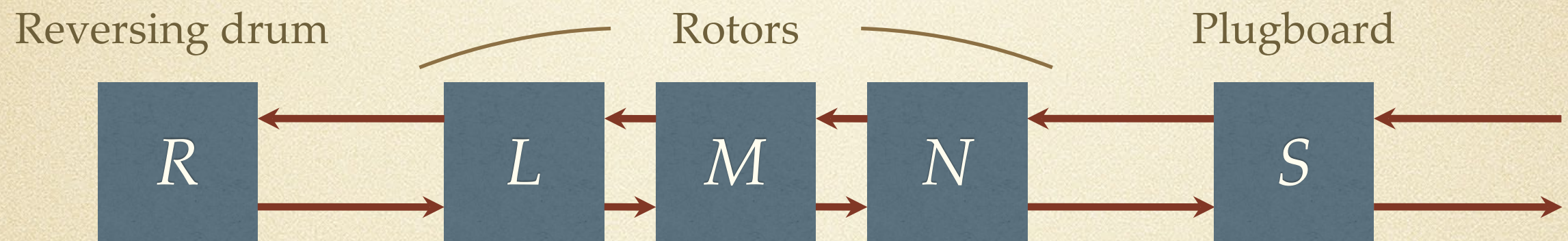
The rotor L :

- Starting position:
Permutation = L
- For the first letter:
Permutation = $\varphi L \varphi^{-1}$
- For the second letter:
Permutation = $\varphi^2 L \varphi^{-2}$

XY means “apply X , then Y ”



Ignoring rotation



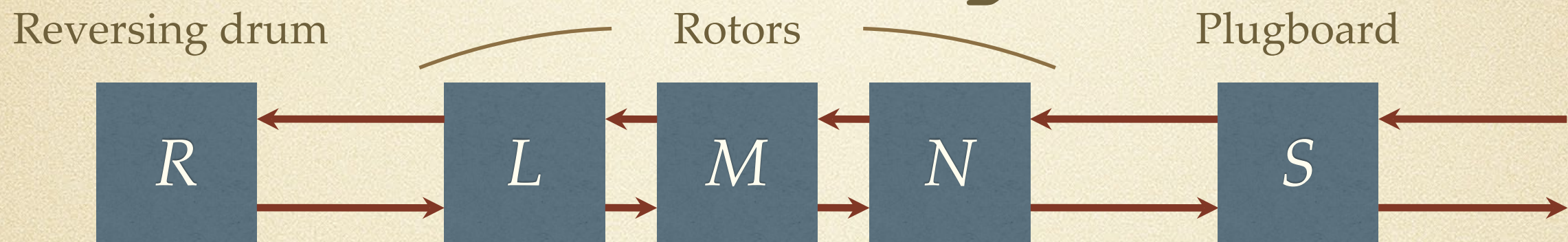
The full encryption permutation:
 $SNMLRL^{-1}M^{-1}N^{-1}S$

Because $R^2 = S^2 = 1$:

$$SNMLRL^{-1}M^{-1}N^{-1}S \cdot SNMLRL^{-1}M^{-1}N^{-1}S = 1$$

Encryption = decryption!

Encrypting a message



$$\varphi = (a \ b \ c \ d \ \dots \ y \ z)$$

For the first few letters, assume cogs M, L do not turn.

First letter:

$$S\varphi N\varphi^{-1}MLRL^{-1}M^{-1}\varphi N^{-1}\varphi^{-1}S$$

Second letter:

$$S\varphi^2 N\varphi^{-2}MLRL^{-1}M^{-1}\varphi^2 N^{-1}\varphi^{-2}S$$

Third letter:

$$S\varphi^3 N\varphi^{-3}MLRL^{-1}M^{-1}\varphi^3 N^{-1}\varphi^{-3}S$$

...

What is kept secret?

Design secrets:

- Design of the machine
- Internal wiring of the rotors

Daily keys:

- Plugboard settings
- Initial position of rotors

Our aim

Follow the Polish cryptographers from the 1930s:

Determine the internal wiring of the rotors!

What do we start with?

- The overall structure of the machine
(but not the rotor wiring)
- Many ciphertexts and corresponding daily keys

The big mistake

Each message was enciphered using different rotor settings

The protocol:

1. Set the rotors according to the daily key
2. Send the 3-letter message key (the new rotor settings)
3. Send the 3-letter message key again
4. Reset the rotors to the message key
5. Send the message

So what?

Every message began with six letters of the form:

x y z x y z

That is:

plaintext letter 1 = plaintext letter 4

plaintext letter 2 = plaintext letter 5

plaintext letter 3 = plaintext letter 6

This was enough to work out how the rotors were wired (and more)!

Building permutations

Let $A, B, C, D, E, F : \{a,b,c,\dots,z\} \rightarrow \{a,b,c,\dots,z\}$ be the encryption functions for positions 1, 2, 3, 4, 5, 6.

These are just permutations!

We do not know A, B, C, D, E or F . But...

AD turns ciphertext letter 1 into ciphertext letter 4
 BE turns ciphertext letter 2 into ciphertext letter 5
 CF turns ciphertext letter 3 into ciphertext letter 6

With enough (~ 80) ciphertext messages, we can completely reconstruct the products AD, BE and CF .

An example

AD : ciphertext $y_1 \mapsto y_4$

BE : ciphertext $y_2 \mapsto y_5$

CF : ciphertext $y_3 \mapsto y_6$

Ciphertext 1: dmqvbn...

Ciphertext 2: vonpuy...

Ciphertext 3: pucfmq...

AD : $d \mapsto v \mapsto p \mapsto f$

BE : $o \mapsto u \mapsto m \mapsto b$

CF : $c \mapsto q \mapsto n \mapsto y$

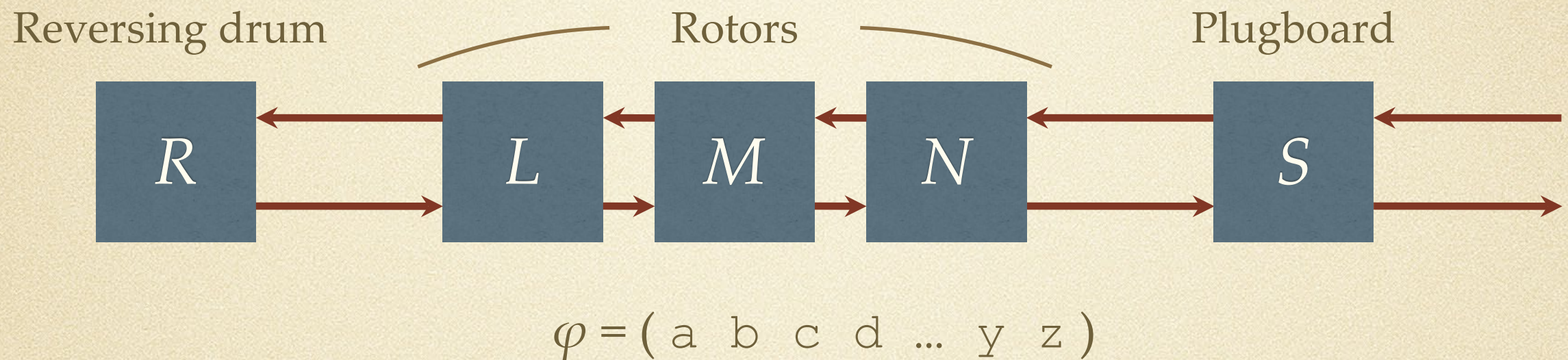
With more messages, we could reconstruct (for example):

AD = (dvpfkxgzyo) (eijmunqlht) (bc) (rw) (a) (s)

BE = (blfqveoum) (hjpswizrn) (axt) (cgy) (d) (k)

CF = (abviktjgfcqny) (duzrehlxwpsmo)

Taking stock



We know AD, BE, CF but not A, B, C, D, E, F individually

$$A = S\varphi N\varphi^{-1}MLRL^{-1}M^{-1}\varphi N^{-1}\varphi^{-1}S$$

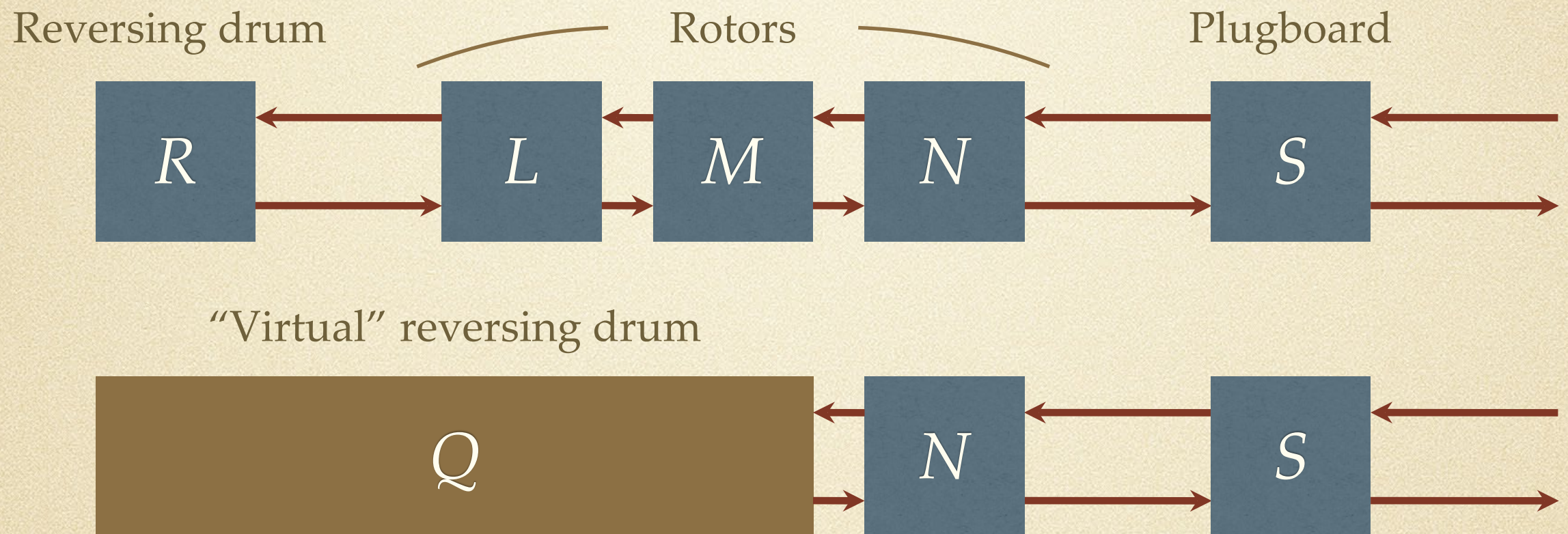
$$B = S\varphi^2 N\varphi^{-2}MLRL^{-1}M^{-1}\varphi^2 N^{-1}\varphi^{-2}S$$

...

$$F = S\varphi^6 N\varphi^{-6}MLRL^{-1}M^{-1}\varphi^6 N^{-1}\varphi^{-6}S$$

We want the internal wiring of the rotors — we want L, M, N

A simplification

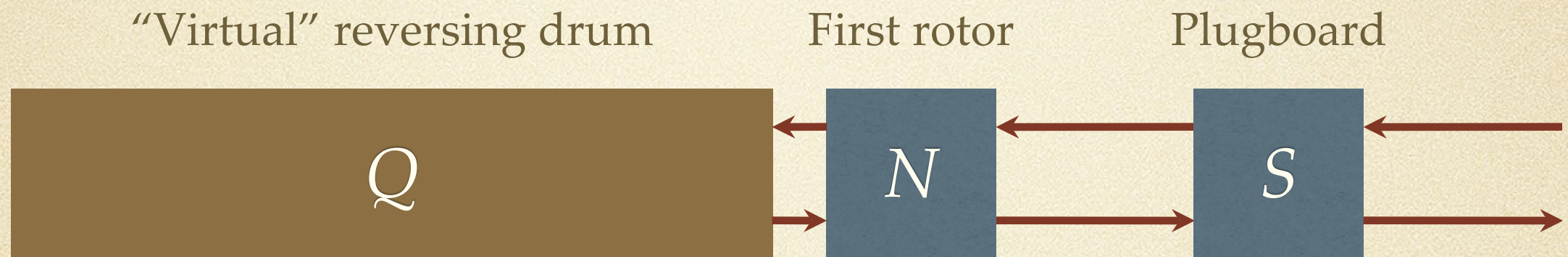


Q behaves just like R :

- Consists of pair swaps that change every letter:
 $Q^2 = 1$, and $Q(x) \neq x$ for all x

Encryption functions $(A, B, \dots, F)$ are like this also!

Taking stock again



$$\varphi = (a \ b \ c \ d \ \dots \ y \ z)$$

Known:

AD , BE , CF , and S

Unknown:

A , B , C , D , E , F , N , Q

Aim:

Deduce N

$$A = S\varphi N\varphi^{-1}Q\varphi N^{-1}\varphi^{-1}S$$

$$B = S\varphi^2 N\varphi^{-2}Q\varphi^2 N^{-1}\varphi^{-2}S$$

...

$$F = S\varphi^6 N\varphi^{-6}Q\varphi^6 N^{-1}\varphi^{-6}S$$

A , B , C , D , E , F and Q :

pair swaps that
change every letter

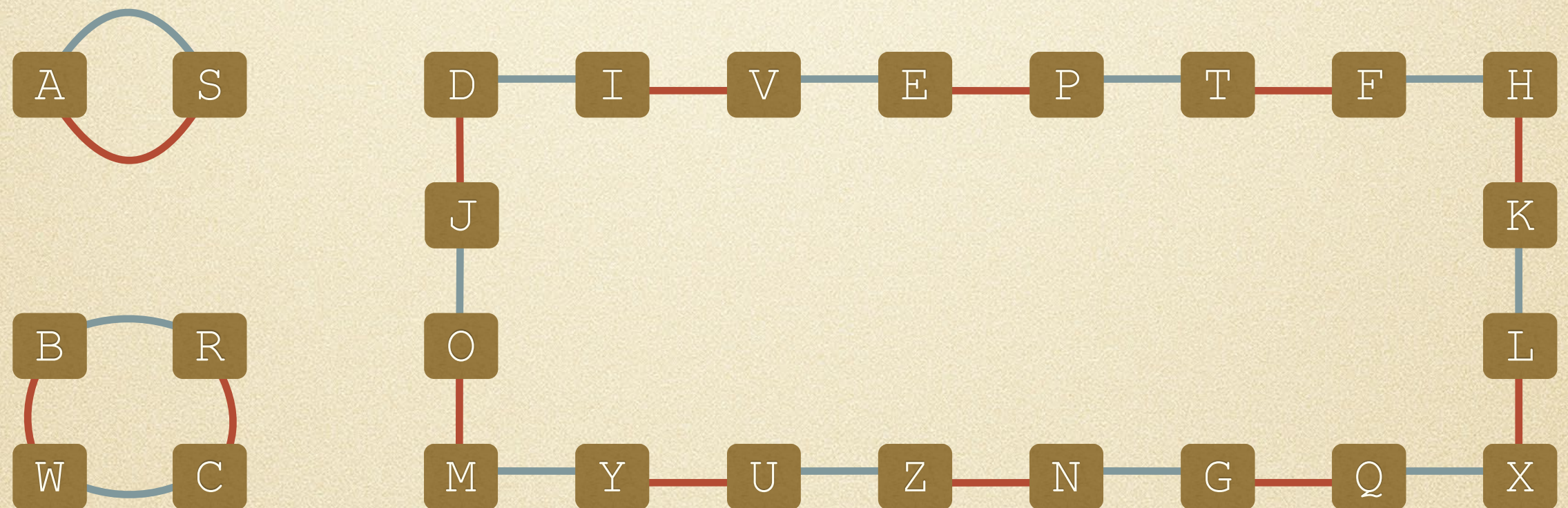
Solving for $A, B, \dots, F$

A and D are pair swaps, with no fixed points:

$A = (as) (br) (cw) (di) (ev) (fh) (gn) (jo) (kl) (my) (pt) (qx) (uz)$

$$D = (as) (bw) (cr) (dj) (ep) (ft) (gq) (hk) (iv) (lx) (mo) (nz) (uy)$$

A and D arrange the alphabet into cycles



Each cycle of this graph has alternating A – D (blue–red) edges

AD : follow blue, then red

$$AD = (a) (s) (bc) (rw) (dvpfkxgzyo) (eijmunqlht)$$

Each cycle σ of this graph produces **two cycles in AD** ,
of the **same length**, walking around σ in **opposite directions**

Question

$BE = (\text{blfqveoum}) (\text{hjpswizrn}) (\text{axt}) (\text{cgy}) (\text{d}) (\text{k})$

How many possible solutions for B, E ?

The graph must have **three cycles**, of lengths 18, 6, 2

There are $9 \times 3 \times 1 = 27$ ways to “**interleave**” the pairs of clockwise/anticlockwise cycles from BE

Only **27 possible solutions** in total!

These are **easily enumerated** by hand!

Solving for A, B, ..., F

$AD = (\text{dvpfkxgzyo}) (\text{eijmunqlht}) (\text{bc}) (\text{rw}) (\text{a}) (\text{s})$

$BE = (\text{blfqveoum}) (\text{hjpswizrn}) (\text{axt}) (\text{cgy}) (\text{d}) (\text{k})$

$CF = (\text{abviktjgfcqny}) (\text{duzrehlxwpsmo})$

20 = $10 \times 2 \times 1$ possibilities for A and D

27 = $9 \times 3 \times 1$ possibilities for B and E

13 possibilities for C and F

$\Rightarrow$ 7020 possible solutions for A, B, C, D, E and F!

This is much less than $(26!)^6 \simeq 4.3 \times 10^{159}$,
and we could enumerate them if we want.

But... what if we knew a little more?

Behavioural knowledge

$AD = (dvpfkxgzyo) (eijmunqlht) (bc) (rw) (a) (s)$

$BE = (blfqveoum) (hjpswizrn) (axt) (cgy) (d) (k)$

$CF = (abviktjgfcqny) (duzrehlxwpsmo)$

What if...

- The operator is known to use message keys of the form $\alpha\alpha\alpha$
(so all six plaintext letters are the same)
- The following ciphertexts have been seen:
xqrgve
nfaqqb
eugimf

Can we solve for A, B, C, D, E and F uniquely?

Repeated plaintext letters

$AD = (dvpfkxgzyo) (eijmunqlht) (bc) (rw) (a) (s)$

$BE = (blfqveoum) (hjpswizrn) (axt) (cgy) (d) (k)$

$CF = (abviktjgfcqny) (duzrehlxwpsmo)$

Ciphertexts: xqrgve, nfaqqb, eugimf

$A: x \mapsto \text{one of } \{e, i, j, m, u, n, q, l, h, t\}$

$B: q \mapsto \text{one of } \{h, j, p, s, w, i, z, r, n\}$

$C: r \mapsto \text{one of } \{a, b, v, i, k, t, j, g, f, c, q, n, y\}$

Only possible plaintexts for xqrgve: $iiiiii, jjjjjj,$
 $nnnnnn$

Only possible plaintexts for nfaqqb: $pppppp, zzzzzz$

Only possible plaintexts for eugimf: $pppppp, zzzzzz$

Repeated plaintext letters

$AD = (dvpfkxgzyo) (eijmunqlht) (bc) (rw) (a) (s)$

$BE = (blfqveoum) (hjpswizrn) (axt) (cgy) (d) (k)$

$CF = (abviktjgfcqny) (duzrehlxwpsmo)$

$xqrgve \mapsto iiiiii / jjjjjj / nnnnnn$

$nfaqqb \mapsto pppppp / zzzzzz$

$eugimf \mapsto pppppp / zzzzzz$

A cannot swap $x \leftrightarrow n$ — inconsistent with message #2!

If A swaps $x \leftrightarrow i$ then it swaps $e \leftrightarrow g$ — inconsistent with #3!

Therefore A swaps $x \leftrightarrow j$.

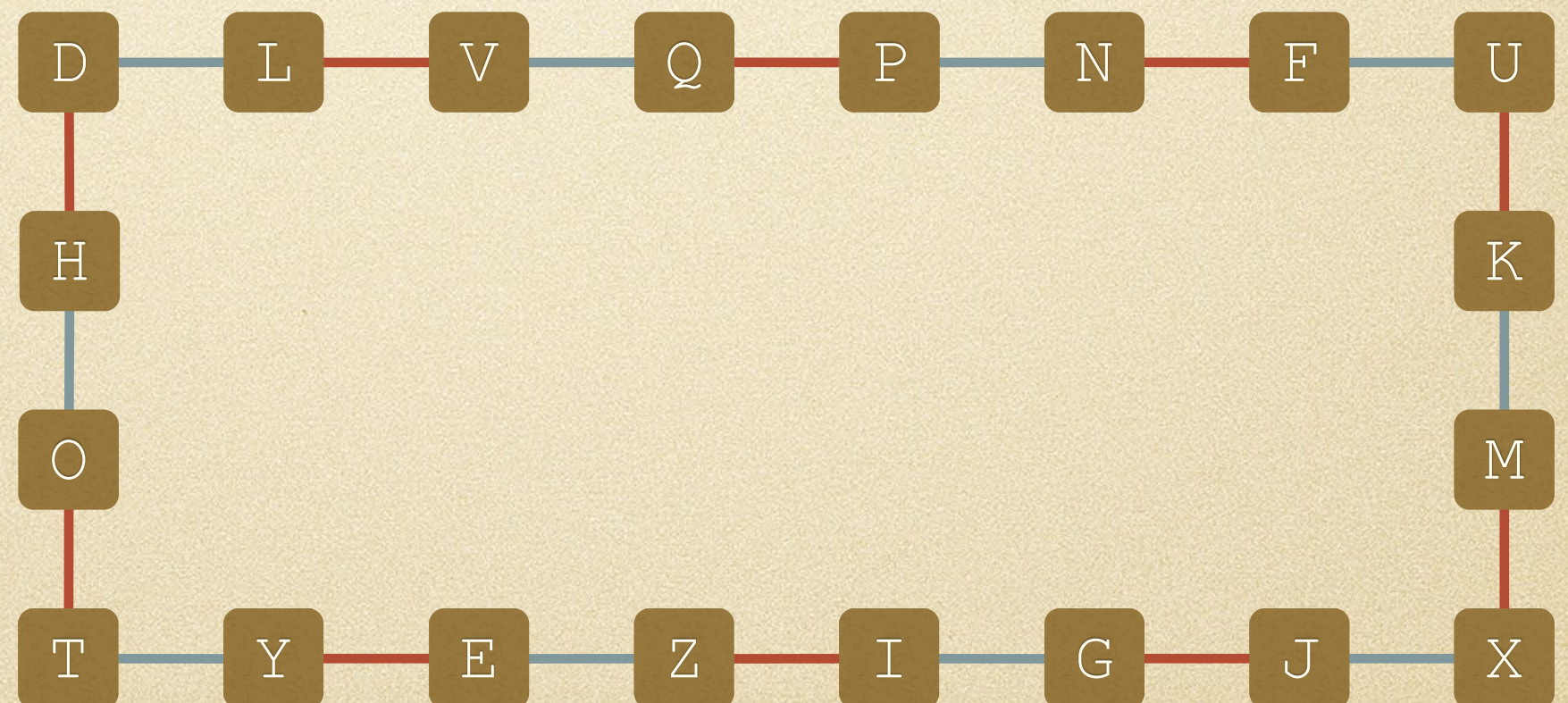
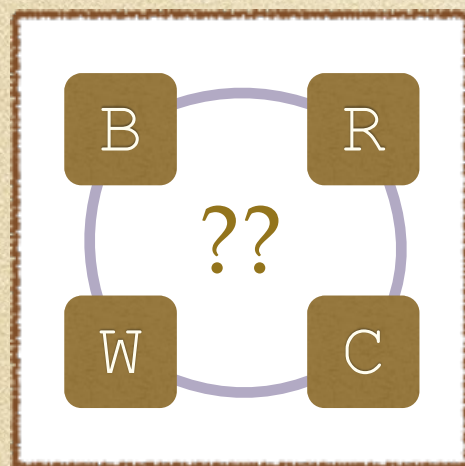
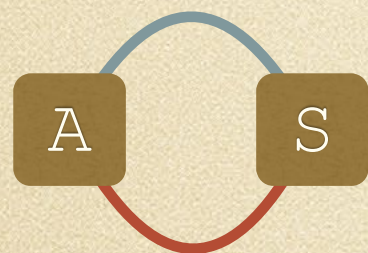
Repeated plaintext letters

AD = (dvpfkxgzyo) (eijmunqlht) (bc) (rw) (a) (s)

BE = (blfqveoum) (hjpswizrn) (axt) (cgy) (d) (k)

CF = (abviktjgfcqny) (duzrehlxwpsmo)

xqrgve $\mapsto$ jjjjjj



Repeated plaintext letters

AD = (dvpfkxgzyo) (eijmunqlht) (bc) (rw) (a) (s)

BE = (blfqveoum) (hjpswizrn) (axt) (cgy) (d) (k)

CF = (abviktjgfcqny) (duzrehlxwpsmo)

xqrgve $\mapsto$ jjjjjj

Repeated plaintext letters

AD = (dvpfkxgzyo) (eijmunqlht) (bc) (rw) (a) (s)

BE = (blfqveoum) (hjpswizrn) (axt) (cgy) (d) (k)

CF = (abviktjgfcqny) (duzrehlxwpsmo)

xqrgve $\mapsto$ jjjjjj

Repeated plaintext letters

$AD = (dvpfkxgzyo) (eijmunqlht) (bc) (rw) (a) (s)$

$BE = (blfqveoum) (hjpswizrn) (axt) (cgy) (d) (k)$

$CF = (abviktjgfcqny) (duzrehlxwpsmo)$

$xqrgve \mapsto jjjjjjj$

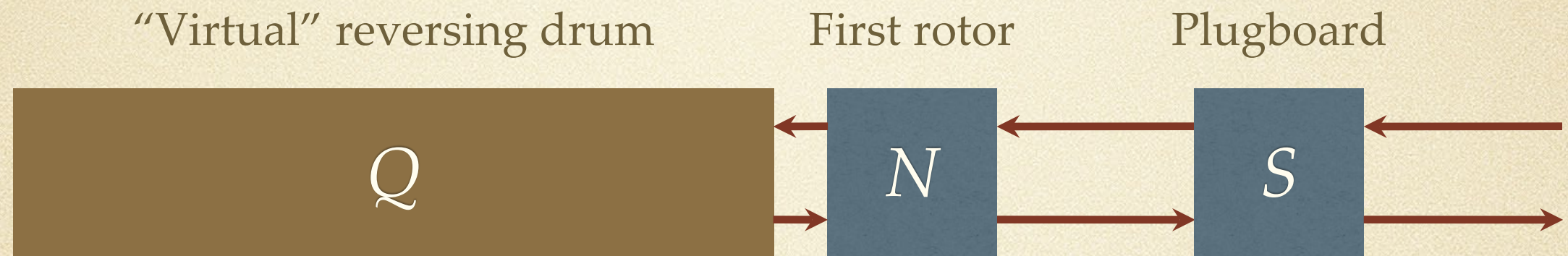
Leaves just 2 options for A and D

Leaves just 3 options for B and E

Specifies C and F precisely!

With a little more work/data, we can completely determine the encryption and decryption permutations $A...F$, despite having no knowledge of the rotor wiring!

Taking stock once more



$$\varphi = (a \ b \ c \ d \ \dots \ y \ z)$$

Known:

A, B, C, D, E, F and S

Unknown:

N, Q

Aim:

Deduce N

$$A = S\varphi N\varphi^{-1}Q\varphi N^{-1}\varphi^{-1}S$$

$$B = S\varphi^2 N\varphi^{-2}Q\varphi^2 N^{-1}\varphi^{-2}S$$

...

$$F = S\varphi^6 N\varphi^{-6}Q\varphi^6 N^{-1}\varphi^{-6}S$$

A, B, C, D, E, F and Q :
pair swaps that
change every letter

Attempting to isolate N

$$A = S\varphi N\varphi^{-1}Q\varphi N^{-1}\varphi^{-1}S$$

$$B = S\varphi^2 N\varphi^{-2}Q\varphi^2 N^{-1}\varphi^{-2}S$$

...

$$F = S\varphi^6 N\varphi^{-6}Q\varphi^6 N^{-1}\varphi^{-6}S$$

$$N\varphi^{-1}Q\varphi N^{-1} = \varphi^{-1}SAS\varphi$$

$$N\varphi^{-2}Q\varphi^2 N^{-1} = \varphi^{-2}SBS\varphi^2$$

...

$$N\varphi^{-6}Q\varphi^6 N^{-1} = \varphi^{-6}SFS\varphi^6$$

Eliminate the unknown Q

$$(1) \quad N\varphi^{-1}Q\varphi N^{-1} = \varphi^{-1}SAS\varphi$$

$$(2) \quad N\varphi^{-2}Q\varphi^2N^{-1} = \varphi^{-2}SBS\varphi^2$$

$$(3) \quad N\varphi^{-3}Q\varphi^3N^{-1} = \varphi^{-3}SCS\varphi^3$$

$$(4) \quad N\varphi^{-4}Q\varphi^4N^{-1} = \varphi^{-4}SDS\varphi^4$$

$$(5) \quad N\varphi^{-5}Q\varphi^5N^{-1} = \varphi^{-5}SES\varphi^5$$

$$(6) \quad N\varphi^{-6}Q\varphi^6N^{-1} = \varphi^{-6}SFS\varphi^6$$

From (1) $\times$ (2):

$$N\varphi^{-1}(Q\varphi^{-1}Q\varphi)\varphi N^{-1} = \varphi^{-1}SAS\varphi^{-1}SBS\varphi^2$$

From (2) $\times$ (3):

$$N\varphi^{-2}(Q\varphi^{-1}Q\varphi)\varphi^2N^{-1} = \varphi^{-2}SBS\varphi^{-1}SCS\varphi^3$$

...

From (5) $\times$ (6):

$$N\varphi^{-5}(Q\varphi^{-1}Q\varphi)\varphi^5N^{-1} = \varphi^{-5}SES\varphi^{-1}SFS\varphi^6$$

Eliminate the unknown Q

$$(7) \quad N\varphi^{-1} (Q\varphi^{-1}Q\varphi) \varphi N^{-1} = \varphi^{-1}SAS\varphi^{-1}SBS\varphi^2$$

$$(8) \quad N\varphi^{-2} (Q\varphi^{-1}Q\varphi) \varphi^2 N^{-1} = \varphi^{-2}SBS\varphi^{-1}SCS\varphi^3$$

$$(9) \quad N\varphi^{-3} (Q\varphi^{-1}Q\varphi) \varphi^3 N^{-1} = \varphi^{-3}SCS\varphi^{-1}SDS\varphi^4$$

$$(10) \quad N\varphi^{-4} (Q\varphi^{-1}Q\varphi) \varphi^4 N^{-1} = \varphi^{-4}SDS\varphi^{-1}SES\varphi^5$$

$$(11) \quad N\varphi^{-5} (Q\varphi^{-1}Q\varphi) \varphi^5 N^{-1} = \varphi^{-5}SES\varphi^{-1}SFS\varphi^6$$

Replace $(Q\varphi^{-1}Q\varphi)$ in (8) using (7):

$$N\varphi^{-1}N^{-1} (\varphi^{-1}SAS\varphi^{-1}SBS\varphi^2) N\varphi N^{-1} = \varphi^{-2}SBS\varphi^{-1}SCS\varphi^3$$

Replace $(Q\varphi^{-1}Q\varphi)$ in (9) using (8):

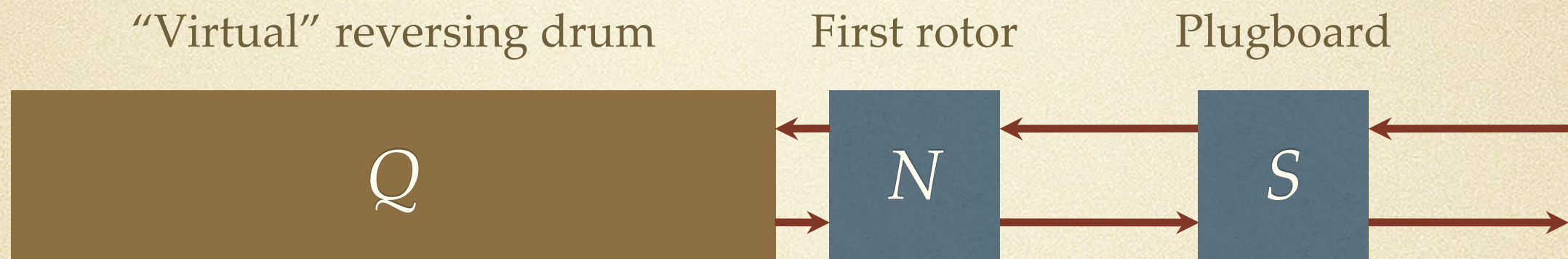
$$N\varphi^{-1}N^{-1} (\varphi^{-2}SBS\varphi^{-1}SCS\varphi^3) N\varphi N^{-1} = \varphi^{-3}SCS\varphi^{-1}SDS\varphi^4$$

...

Replace $(Q\varphi^{-1}Q\varphi)$ in (11) using (10):

$$N\varphi^{-1}N^{-1} (\varphi^{-4}SDS\varphi^{-1}SES\varphi^5) N\varphi N^{-1} = \varphi^{-5}SES\varphi^{-1}SFS\varphi^6$$

Where are we at?



$$\varphi = (a \ b \ c \ d \ \dots \ y \ z)$$

Known: A, B, C, D, E, F and S Aim: Deduce N

Unknown: N, Q

$$\begin{aligned} (12) \quad N\varphi^{-1}N^{-1}(\varphi^{-1}SAS\varphi^{-1}SBS\varphi^2)N\varphi N^{-1} &= \varphi^{-2}SBS\varphi^{-1}SCS\varphi^3 \\ (13) \quad N\varphi^{-1}N^{-1}(\varphi^{-2}SBS\varphi^{-1}SCS\varphi^3)N\varphi N^{-1} &= \varphi^{-3}SCS\varphi^{-1}SDS\varphi^4 \\ (14) \quad N\varphi^{-1}N^{-1}(\varphi^{-3}SCS\varphi^{-1}SDS\varphi^4)N\varphi N^{-1} &= \varphi^{-4}SDS\varphi^{-1}SES\varphi^5 \\ (15) \quad N\varphi^{-1}N^{-1}(\varphi^{-4}SDS\varphi^{-1}SES\varphi^5)N\varphi N^{-1} &= \varphi^{-5}SES\varphi^{-1}SFS\varphi^6 \end{aligned}$$

More simplifications

$$(12) \quad N\varphi^{-1}N^{-1} (\varphi^{-1}SAS\varphi^{-1}SBS\varphi^2) N\varphi N^{-1} = \varphi^{-2}SBS\varphi^{-1}SCS\varphi^3$$

$$(13) \quad N\varphi^{-1}N^{-1} (\varphi^{-2}SBS\varphi^{-1}SCS\varphi^3) N\varphi N^{-1} = \varphi^{-3}SCS\varphi^{-1}SDS\varphi^4$$

$$(14) \quad N\varphi^{-1}N^{-1} (\varphi^{-3}SCS\varphi^{-1}SDS\varphi^4) N\varphi N^{-1} = \varphi^{-4}SDS\varphi^{-1}SES\varphi^5$$

$$(15) \quad N\varphi^{-1}N^{-1} (\varphi^{-4}SDS\varphi^{-1}SES\varphi^5) N\varphi N^{-1} = \varphi^{-5}SES\varphi^{-1}SFS\varphi^6$$

These are equations in $N\varphi^{-1}N^{-1}$.

Let $X = N\varphi^{-1}N^{-1}$:

$$(12) \quad X\alpha X^{-1} = \beta$$

$$(13) \quad X\beta X^{-1} = \gamma$$

$$(14) \quad X\gamma X^{-1} = \delta$$

$$(15) \quad X\delta X^{-1} = \varepsilon$$

... where:

$$\alpha = \varphi^{-1}SAS\varphi^{-1}SBS\varphi^2$$

$$\beta = \varphi^{-2}SBS\varphi^{-1}SCS\varphi^3$$

$$\gamma = \varphi^{-3}SCS\varphi^{-1}SDS\varphi^4$$

$$\delta = \varphi^{-4}SDS\varphi^{-1}SES\varphi^5$$

$$\varepsilon = \varphi^{-5}SES\varphi^{-1}SFS\varphi^6$$

$\alpha, \beta, \gamma, \delta, \varepsilon$ are all **computable!**

Conjugation

Given two permutations U and V , what does UVU^{-1} look like?

Conjugation

Given two permutations U and V , what does UVU^{-1} look like?

The permutation UVU^{-1} has the same cycle structure as V , but the elements have been relabelled.

$$U = (afhgde) (bc) \quad U^{-1} = (adeghf) (bc)$$

$$V = (acf) (bhgde) \\ UVU^{-1} = (dba) (cfheg)$$

Rewrite using
permutation U^{-1}

$$UVU^{-1} = (dba) (cfheg) \\ V = (acf) (bhgde)$$

Rewrite using
permutation U

Solving for X

Suppose we have:

$A = (as) (br) (cw) (di) (ev) (fh) (gn) (jo) (kl) (my) (pt) (qx) (uz)$

$B = (ay) (bj) (ct) (dk) (ei) (fn) (gx) (hl) (mp) (ow) (qr) (su) (vz)$

$C = (ax) (bl) (cm) (dg) (ei) (fo) (hv) (ju) (kr) (np) (qs) (tz) (wy)$

$D = (as) (bw) (cr) (dj) (ep) (ft) (gq) (hk) (iv) (lx) (mo) (nz) (uy)$

$E = (ac) (bp) (dk) (ez) (fh) (gt) (io) (jl) (ms) (nq) (rv) (uw) (xy)$

$F = (aw) (bx) (co) (df) (ek) (gu) (hi) (jz) (lv) (mq) (ns) (py) (rt)$

with plugboard settings: $S = (ap) (bl) (cz) (fh) (jk) (qu)$

Then:

$$\alpha = \varphi^{-1} S A S \varphi^{-1} S B S \varphi^2 = (aepftybsnikod) (rhcgzmuvgwlijx)$$

$$\beta = \varphi^{-2} S B S \varphi^{-1} S C S \varphi^3 = (akjcevzydlwnu) (smtfhqibxopgr)$$

$$\gamma = \varphi^{-3} S C S \varphi^{-1} S D S \varphi^4 = (aqvloikgnwbmc) (puzftjryehxds)$$

$$\delta = \varphi^{-4} S D S \varphi^{-1} S E S \varphi^5 = (agxymsfvceo jt) (bqzklrpwnhiud)$$

$$\varepsilon = \varphi^{-5} S E S \varphi^{-1} S F S \varphi^6 = (ampefiltjbcrz) (dvhnyoqugkwsx)$$

Solving for X

Equations: $X\alpha X^{-1} = \beta$, $X\beta X^{-1} = \gamma$, $X\gamma X^{-1} = \delta$, $X\delta X^{-1} = \varepsilon$

$\alpha =$ (aepftybsnikod) (rhcgzmuqvqljx)

$\beta =$ (akjcevzydlwnu) (smtfhqibxopgr)

$\gamma =$ (aqvloikgnwbmc) (puzftjryehxds)

$\delta =$ (agxymsfvceoht) (bqzklrpwnhiud)

$\varepsilon =$ (ampefilitjbcrz) (dvhnyoqugkwsx)

Solving for X

Equations: $X\alpha X^{-1} = \beta$, $X\beta X^{-1} = \gamma$, $X\gamma X^{-1} = \delta$, $X\delta X^{-1} = \varepsilon$

$\alpha =$ (aepftybsnikod) (rhcgzmuvgwlijx)

$\beta =$ (akjcevzydlwnu) (smtfhqibxopgr)

$\gamma =$ (aqvloikgnwbmc) (puzftjryehxds)

$\delta =$ (agxymsfvceoht) (bqzklrpwnhiud)

$\varepsilon =$ (ampefilitjbcrz) (dvhnyoqugkwsx)

Solving for X

Equations: $X\alpha X^{-1} = \beta$, $X\beta X^{-1} = \gamma$, $X\gamma X^{-1} = \delta$, $X\delta X^{-1} = \varepsilon$

$\alpha =$ (aepftybsnikod) (rhcgzmuvqwljx)

$\beta =$ (akjcevzydlwnu) (smtfhqibxopgr)

$\gamma =$ (aqvloikgnwbmc) (puzftjryehxds)

$\delta =$ (agxymsfvceo jt) (bqzklrpwnhiud)

$\varepsilon =$ (ampefilitjbcrz) (dvhnyoqugkwsx)

This gives $X^{-1} =$ (ayuricxqmgovskedzplfwtnjhb)

Therefore: $X =$ (abhjntwflpzdeksvogmqxciruy)

Solving for N

Finally: $N\varphi^{-1}N^{-1} = X = (\text{abhjntwflpzdekstvogmqxciruy})$

$N\varphi^{-1}N^{-1} = (\text{abhjntwflpzdekstvogmqxciruy})$
 $\varphi^{-1} = (\text{azyxwvutsrqponmlkjihgfedcb})$  N is the relabel rule

$N = (\text{a}) (\text{bzqhy}) (\text{cftvlsmieoknwu}) (\text{dpr}) (\text{gjx})$

A different way to write φ^{-1} gives a different N .

$N\varphi^{-1}N^{-1} = (\text{abhjntwflpzdekstvogmqxciruy})$
 $\varphi^{-1} = (\text{bazyxwvutsrqponmlkjihgfedc})$

$N = (\text{ab}) (\text{cgkoltwvmjy}) (\text{dqifu}) (\text{epsnxh zr})$

Actually... there are **26 possible solutions**,
corresponding to the 26 ways of writing φ^{-1} beneath $N\varphi^{-1}N^{-1}$.

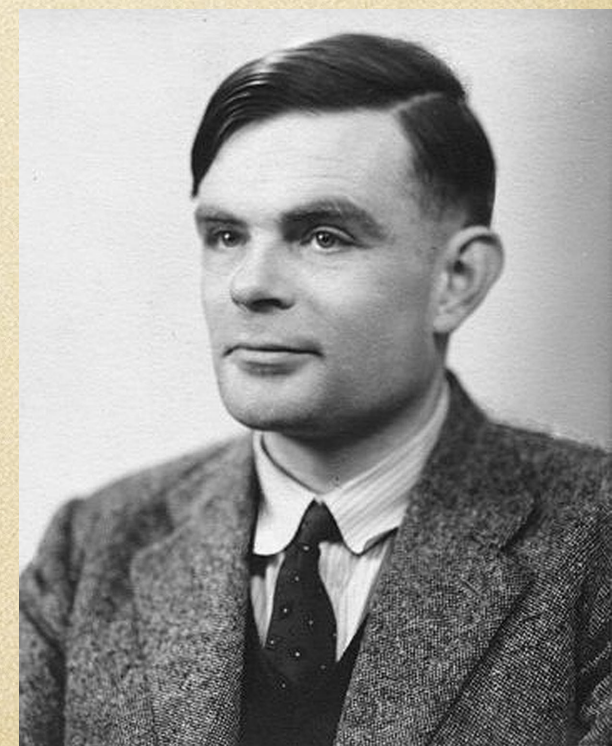
This is fine—there are 26 ways to label the same rotor!

The other two rotors

- This gives us rotor *N*, but not rotors *M* and *L*
- But... the Germans made the cipher harder by occasionally reordering the three rotors
- This means, over time, **all three rotors** became vulnerable to this cryptanalysis!

Final comments

- The Germans changed their machines at times. Breaking Enigma was a cat-and-mouse game.
- The work discussed here was done by the **Polish**, before the war. Read more about **Marian Rejewski**, **Henryk Zygalski**, **Jerzy Rozycki** (died in war).
- In 1938 the Germans upgraded to **five rotors**, not three.
- Read also about **Alan Turing**, Bletchley Park, and the development of the **cryptological** bombs.



Secure stream ciphers

What do we want from a keystream?

1. It should be **unpredictable**.

- New keystream bits are **difficult to deduce**, even if many past keystream bits are known
- Guards against **known plaintext attacks**

2. It should be **reproducible**.

- Generated deterministically from a **small seed**

Random numbers

Cryptographers often need “high quality” random numbers (or bits), for things like key generation, one-time pads, ...

Perfectly unpredictable randomness

- Not reproducible (by design)
- Typically requires physical input
- Expensive and/or slow to produce

Pseudo-randomness

Computers use deterministic algorithms to produce what *looks* like a stream of random numbers

- Generated **deterministically** from a small seed
- Fast and cheap
- For cryptographic applications, these should also be **unpredictable** unless the seed is known

This raises the same problem as for stream ciphers!

Find a deterministic (i.e., reproducible)
but unpredictable source of data

Cryptographically secure pseudo-random number generators

What do we mean by “unpredictable”?

- Suppose we are generating random bits $x_1, x_2, x_3, \dots \in \mathbb{Z}_2$
- Given a consecutive sequence of bits $x_1, x_2, \dots, x_k$, there is no polynomial-time algorithm that can predict x_{k+1} with probability significantly greater than $\frac{1}{2}$

Cryptographically secure pseudo-random number generators

We would typically formalise this further. For example:

- Let n be size of the seed in bits (e.g., if the seed is N then $n \simeq \log_2 N$)
- We might allow ourselves to ignore “enormously long” sequences:
Restrict to $k \leq \text{polynomial in } n$
- We might allow ourselves to ignore a small number of “weak seeds”:
Require that the fraction of seeds we ignore $\rightarrow 0$ as $n \rightarrow \infty$
- We might define “significantly greater than $\frac{1}{2}$ ” more precisely:
 $\Pr(\text{correctly predict } x_{k+1}) \leq \frac{1}{2} + 1/n^t$ for all $t > 0$
- We might assume some core mathematical problem (e.g., factoring) has no probabilistic polynomial-time algorithm that is correct “significantly often”

Formalising such results **precisely** is a longer discussion for another time.

Blum-Blum-Shub

Seed:

Choose large primes p and q , where $p, q \equiv 3 \pmod{4}$

Compute $n = pq$

Choose integer s with $0 < s < n$ and $\gcd(s, n) = 1$

Sequence:

$$x_0 = s^2 \bmod n$$

$$x_i = x_{i-1}^2 \bmod n$$

Output bit z_i is the least significant bit of x_i (even vs odd)

Example:

$$p = 7, q = 19$$

$$n = 133$$

$$s = 2$$

Sequence x_i : 4 16 123 100 25 93 4 16 ...

Output bits : 0 0 1 0 1 1 0 0 ...

In general, the period relates to Carmichael's λ -function

Blum-Blum-Shub

Blum-Blum-Shub is provably unpredictable, assuming that integer factorisation is intractably hard.

It is also expensive (i.e., slow).